

FIIT STU

Dátové štruktúry a algoritmy

Filip Remšík

ID:110883

2020/2021

Vyhľadavanie v dynamických množinách

Cvičenie štvrtok 11:00

Moje Implementácie

AVL stromy

Typ binárneho stromu, pri ktorom dochádza k vyvažovaniu ak absolútna hodnota výšok dvoch susedných uzlov je väčšia ako 2. Na vyvažovanie sa používajú 4 typy rotácií: Pravá, ľavá a ich kombinácie. Ak je správne vyvážený, tak je vyhľadávanie rýchle.

```
int search(Node* node, int key) {
    if (node == NULL) {
        return(0);
    }
    if (node->key == key) {
        return(0);
    }
    if (key < node->key)
        search(node->left, key);
    else if (key > node->key)
        search(node->right, key);
    return(0);
}
```

Zložitosť: $O(\log n)$ -platí pre vkladanie aj vyhľadávanie.

HASH Funkcia

Vo funkcii sa sčítavajú ascii hodnoty znaku prenasobené ich poradím zväčšeným o 7. Na záver sa kľúč prenasobí veľkosťou reťazca a výsledný kľúč sa získa zo zvyšku po delení veľkosťou poľa.

```
for (int i = 1; i <= strlen(word); i++) {
    key += ((word[i]) * (i + 7));
}
key = (key * (strlen(word))) % count;
```

Zložitosť: $O(1)$ =ak vstupný údaj ešte nie je v tabuľke, inak je $O(n)$.

Prevzaté implementácie

Červeno-čierne stromy

Typ binárneho stromu, pričom uzlom je pridelená farba a uzol jednej farby musí mať susedov rozdielnej farby. Oproti AVL stromom si musia teda uchovávať farbu, AVL majú zas vzdialenosť od listov. Vkladanie doň je rýchlejšie v porovnaní s AVL, ale pri vyhľadávaní si konkurujú.

```
enum type { RED, BLACK };

struct Node
{
    int data;
    struct Node* left;
    struct Node* right;
    struct Node* parent;
    enum type color;
};
```

Zložitosť: $O(\log n)$ -platí pre vkladanie aj vyhľadávanie.

Zdroj: <https://github.com/amitbansal7/Data-Structures-and-Algorithms/blob/master/9.Red-Black-tree/RedBlackTrees.c>

HASH Funkcia

Jednoduchá funkcia, funguje len na princípe delenia veľkosťou poľa, ak dôjde ku kolízii jednoducho ju odignoruje.

```
int hashCode(int key)
{
    return (key % capacity);
}
```

Zdroj: <https://www.sanfoundry.com/c-program-implement-hash-tables/>

Testovanie

Stromy

AVL strom 10 mil. položiek

```
Inserting AVL: 50.608000 seconds
Search AVL : 5.359000 seconds
```

RB strom 10 mil.položiek

```
Inserting RB: 2.916000 seconds
Search RB : 5.573000 seconds
```

Vkladanie 10 mil. Položiek,náhodná generácia

```
Inserting AVL: 31.635000 seconds
Inserting RB : 21.540000 seconds
```

Hash

Vkladanie 10 mil. čísiel do tabuľky rovnakej veľkosti

```
Insert to copy hash 0.510000 seconds  
Insert to my hash 6.394000 seconds
```

Môj program na rozdiel od prevzatého pri kolízii hľadá voľné miesto, preto je jeho čas pomalší.

Vkladanie 10 mil. čísiel do tabuľky veľkosti 20 mil.

```
Insert to copy hash 0.598000 seconds  
Insert to my hash 51.298000 seconds
```

Aj keď počet vstupných údajov bol rovnaký, vkladanie bolo pomalšie z dôvodu, že bolo potrebné najskôr alokovať väčšiu pamäť.